Title of the Project

Protocol Compliance Verification of Multi-Standard Communication Controllers Using Assertion-Based Verification in SystemVerilog

Table of Contents

Abstract

This report presents the design, implementation, and assertion-based verification of a multi-standard serial communication subsystem supporting three widely used embedded protocols — UART, SPI (Mode 0), and I²C (7-bit addressing, write mode). The project centres on a 5-state finite state machine (FSM) based arbitration module, referred to as the protocol switching manager, which governs runtime switching between the three protocol controllers. To rigorously verify both the individual protocol controllers and the composite temporal behaviour of the system across protocol boundaries, a library of 23 SystemVerilog Assertions (SVAs) was developed and embedded directly within the RTL source files. These assertions capture cycle-accurate timing properties of each protocol as well as switching-fabric correctness constraints such as mutual exclusion, ordered switching, and no-overlap-during-transition.

To validate the diagnostic strength of the assertion library, four synthesizable fault-injection wrapper modules were built to deliberately corrupt the stop bit in UART, the chip-select signalling in SPI, the SDA data-line stability in I²C, and the mutual-exclusion property of the switching manager. The assertion library detected all injected faults within 1–7 clock cycles with zero false negatives. A unified integration testbench exercised three complete protocol-switching cycles and performed 25 individual correctness checks, with zero assertion violations recorded against the golden RTL. Functional coverage was collected using a covergroup comprising 26 bins, achieving 100% overall coverage, with all six inter-protocol transition directions exercised and confirmed using simulation-based transition counters.

The complete design and verification environment was written in IEEE 1800-2017 SystemVerilog and simulated using Xilinx Vivado 2025.2 / XSim. The results demonstrate that assertion-based verification, when combined with structural design guarantees and systematic fault injection, provides a robust and reusable methodology for verifying multi-protocol, runtime-reconfigurable communication subsystems — a verification challenge that traditional directed-test simulation addresses only partially.

1. Introduction

Serial communication protocols form the backbone of modern embedded systems and System-on-Chip (SoC) designs. Among the most widely used are UART (Universal Asynchronous Receiver/Transmitter), SPI (Serial Peripheral Interface), and I²C (Inter-Integrated Circuit). Each of these protocols occupies a distinct point in the speed-versus-complexity design space: UART offers asynchronous communication with minimal hardware overhead, SPI offers high-throughput synchronous data transfer through a dedicated point-to-point interface, and I²C offers multi-device addressing over a minimal two-wire bus for clocked peripheral devices.

As SoC designs grow increasingly complex, a single communication interface block is often required to support more than one protocol so that an appropriate controller can be selected for whichever peripheral is attached at runtime. This multi-protocol, runtime-switching paradigm introduces a qualitatively new verification challenge. Verifying each protocol controller in isolation is no longer sufficient: the arbitration logic that governs switching between protocols must itself be verified for correctness, and the composite temporal behaviour of the system across protocol boundaries must also be verified. The conventional approach of directed-test simulation validates such systems only partially, since it is difficult to anticipate every possible switching scenario and every possible fault mode purely through hand-written test vectors.

SystemVerilog Assertions (SVAs), defined in the IEEE 1800 standard and refined in subsequent revisions, provide an executable specification language capable of expressing temporal properties directly alongside RTL source code. Assertions are checked automatically at every clock edge during simulation, and any violation generates a diagnostic message identifying the violated property, the module in which it occurs, and the simulation time at which the violation was observed. This project adopts an assertion-first verification methodology built around SVAs to address the verification challenges introduced by runtime protocol switching.

1.1 Contributions of this Project

This project contributes the following to the body of work on multi-protocol controller verification:

1. Three synthesizable RTL protocol controllers — UART (8N1 framing), SPI (Mode 0), and an I²C write master — together with a five-state FSM-based protocol-switching manager, all implemented in IEEE 1800-2017 SystemVerilog.

2. An SVA library of 23 assertions covering protocol-specific temporal properties as well as switching-fabric constraints, namely mutual exclusion, switch-after-done ordering, and no-overlap-during-transition.

3. Four synthesizable fault-injection wrappers, accompanied by a full assertion escape analysis that demonstrates zero false negatives across all injected fault classes.

4. A unified integration testbench comprising 25 individual correctness checks and a covergroup-based functional coverage model that achieves 100% coverage across all 26 defined bins.

2. Literature Survey

The verification methodology adopted in this project builds upon a well-established body of literature on assertion-based verification (ABV) and protocol-specific formal and simulation-based checking. The IEEE 1800 standard [1] defines the syntax and semantics of SystemVerilog, including the SystemVerilog Assertion (SVA) constructs used throughout this project's RTL and verification environment.

2.1 Foundations of Assertion-Based Verification

Foster, Krolnik, and Lacey [2] developed the foundational taxonomy of assertion-based verification, classifying design properties into safety properties, which must always hold, and liveness properties, which must eventually hold. They further demonstrated the role that such properties play in SoC sign-off flows, establishing the conceptual basis for treating assertions as first-class, executable design specifications rather than as after-the-fact documentation.

Vijayaraghavan and Ramanathan [3] provided a complete and practical treatment of SVA syntax and semantics, including a detailed explanation of the overlapping implication operator (|->) and the non-overlapping implication operator (|=>). Both operators are used extensively throughout the assertion library developed in this project, since almost every temporal property relating a protocol's FSM state to its output signals is expressed using one of these two implication forms.

Bening and Foster [4] formalized a catalogue of assertion patterns for verifiable RTL design. A number of the assertions used in this project — in particular the stop-bit and chip-select correctness checks — correspond directly to patterns described in their work. Bergeron, Cerny, Hunter, and Nightingale [5] formalized the methodology for functional coverage collection using SystemVerilog covergroup and cover directive constructs; the coverage strategy adopted in this project, comprising per-signal coverpoints and a cross-coverage bin for protocol-selection versus active-protocol state, is directly informed by these principles.

2.2 Protocol-Specific Verification

With regard to protocol-specific verification, Shroff, Shah, and Jotwani [6] applied formal property checking to the I²C protocol and derived temporal properties similar to the SDA-stability and START/STOP-condition assertions developed in this project. Kumar and Singh [7] demonstrated SVA-based verification of SPI protocol controllers; however, their work addressed the SPI protocol in isolation, without runtime arbitration between multiple protocols and without any fault-injection framework to validate the assertions themselves.

The closest parallel to this project is the work of Huang et al. [8], who presented coverage-driven verification of multi-protocol interface controllers. That work, however, did not consider runtime switching between protocols, did not include a unified arbitration module, and did not incorporate synthesizable fault injection or assertion escape analysis — all of which are key elements of the present project. Spear [9] and Navabi [10] provide broader background references on SystemVerilog-based verification methodology and hardware description language modelling respectively, both of which informed the general RTL coding style adopted for the protocol controllers.

2.3 Gap Addressed by this Project

Existing protocol verification approaches primarily verify UART, SPI, and I²C as independent interfaces. Limited work addresses assertion-based verification of a unified controller supporting runtime protocol switching, integrated fault injection, and functional coverage within a single verification framework

Problem Statement

Modern SoC communication subsystems are frequently required to support more than one serial protocol within a single interface block, with the active protocol selected at runtime according to the peripheral currently attached. This introduces two compounding verification difficulties. First, each individual protocol controller — UART, SPI, and I²C — must independently satisfy its own protocol-specific timing and framing rules. Second, and more critically, the arbitration logic that switches between these controllers must be verified to guarantee mutual exclusion (only one protocol controller may be active at any time), ordered switching (a switch request must not interrupt a protocol transfer that is still in progress), and absence of signal overlap during the transition between one protocol and the next.

Directed-test simulation, the conventional verification approach for such systems, exercises only the specific scenarios that a test-writer has anticipated in advance. It does not offer a systematic guarantee that every possible switching sequence, every possible fault mode, and every possible protocol-boundary interaction has been exercised and checked. There is, in addition, no straightforward means by which a directed test alone can demonstrate that the checking mechanism itself would actually detect a defect if one were present in the design — that is, directed tests alone cannot demonstrate the absence of false negatives in the verification environment.

The problem addressed by this project is therefore twofold: (i) to design and implement a synthesizable, runtime-reconfigurable multi-protocol communication subsystem supporting UART, SPI, and I²C with a dedicated FSM-based switching manager, and (ii) to develop and validate an assertion-based verification environment — supported by systematic fault injection and functional coverage — that can verify both the individual protocol controllers and the composite correctness of the runtime switching fabric, while also demonstrating, through deliberate fault injection, that the verification environment itself is capable of catching real design defects.

4. Objectives

The specific objectives pursued in this project are as follows:

1. To design and implement three synthesizable RTL protocol controllers — a UART transmitter with 8N1 framing, an SPI master operating in Mode 0, and an I²C master supporting 7-bit addressed write transactions — in IEEE 1800-2017 SystemVerilog.

2. To design a five-state FSM-based protocol switching manager capable of arbitrating runtime switching between the three protocol controllers while structurally guaranteeing mutual exclusion of the protocol enable signals.

3. To develop a library of SystemVerilog Assertions capturing the cycle-accurate temporal behaviour of each protocol controller as well as the correctness constraints of the switching fabric — mutual exclusion, switch-after-done ordering, and no-overlap-during-transition.

4. To construct synthesizable fault-injection wrapper modules that deliberately corrupt specific protocol signals, and to verify — through assertion escape analysis — that the assertion library detects every injected fault with zero false negatives.

5. To build a unified integration testbench that exercises complete protocol-switching cycles end-to-end and reports pass/fail status for a comprehensive set of individual correctness checks.

6. To collect functional coverage using a covergroup-based model and to demonstrate 100% coverage closure across all defined coverpoints and cross-coverage bins, including all six possible inter-protocol transition directions.

7. To simulate and validate the complete design and verification environment using an industry-standard simulation tool (Xilinx Vivado / XSim) and to document the resulting waveforms, assertion logs, and coverage reports.

5. System Design

The verification environment developed in this project is organized into four distinct layers, illustrated in Fig. 5.1: the RTL Design Layer, the Assertion and Coverage Layer, the Testbench Layer, and the Fault Injection Layer. This layered organization keeps the design (what the hardware does), the specification (what the hardware must do, expressed as assertions), the stimulus (what is driven at the testbench), and the fault model (deliberately broken variants of the design) cleanly separated, while still allowing them to be simulated together within a single unified environment.

5.1 RTL Design Layer

The RTL Design Layer comprises four synthesizable modules: uart_tx, spi_master, i2c_master, and protocol_switch. All four modules are implemented as always_ff-based finite state machines, with their output signals decoded in a single always_comb block per module to avoid multiple-driver conflicts on shared output nets.

5.2 Assertion and Coverage Layer

The Assertion and Coverage Layer consists of SVA assert property statements co-located within each RTL module and enclosed within `ifndef SYNTHESIS guards so that the assertions are excluded from synthesis while remaining fully active in simulation. A dedicated coverage_collector module, instantiated within the testbench, samples a covergroup on every rising clock edge and reports detailed bin-level coverage results at the end of simulation.

5.3 Testbench Layer

The Testbench Layer includes three unit-level testbenches — one each for the UART, SPI, and I²C controllers — together with a unified integration testbench that instantiates all four RTL modules simultaneously and drives three complete protocol-switching cycles.

5.4 Fault Injection Layer

The Fault Injection Layer consists of four wrapper modules that instantiate the golden (correct) RTL and override specific output signals using continuous assign statements. This produces synthesizable, but deliberately incorrect, design variants that are used to validate the diagnostic strength of the assertion library.

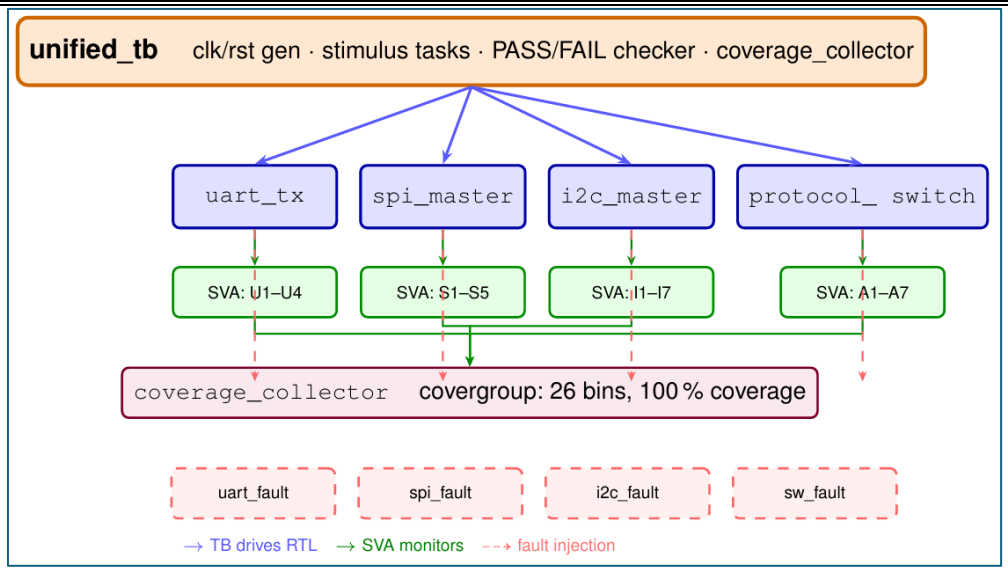


Fig. 5.1. Layered verification architecture, showing the unified testbench driving the four RTL modules (uart_tx, spi_master, i2c_master, protocol_switch), each instrumented with its own set of SVAs (U1–U4, S1–S5, I1–I7, A1–A7), a shared coverage_collector sampling a 26-bin covergroup, and four fault-injection wrapper modules feeding deliberately corrupted signal variants into the same checking infrastructure.

5.5 Protocol Selection Encoding

Table 5.1 defines the two-bit encoding shared by both the testbench input signal proto_sel and the switching-manager output signal active_proto. The value 2'b11 denotes the idle condition in which no protocol controller is active.

proto_sel[1:0]	active_proto[1:0]	Selected Protocol
2'b00	2'b00	UART
2'b01	2'b01	SPI
2'b10	2'b10	I²C
2'b11	2'b11	None (idle)

Table 5.1. Protocol selection encoding shared by proto_sel and active_proto.

6. RTL Implementation

This section describes the internal design of each of the three synthesizable protocol controllers implemented in this project. Each controller is realized as a clocked finite state machine (always_ff) with a companion always_comb block that decodes the current state into the protocol's output signals.

6.1 UART Transmitter (uart_tx)

The UART transmitter implements standard 8N1 framing: one LOW start bit, eight data bits transmitted LSB-first, and one HIGH stop bit. The baud rate is parameterised by CLKS_PER_BIT = ⌊f_clk / f_baud⌋, allowing the same RTL to be retargeted to different baud rates purely by changing a parameter. The transmitter's finite state machine, given in Algorithm 6.1, cycles through four states — IDLE, START, DATA, and STOP.

Algorithm 1 UART Transmitter FSM

Input: `tx_start, tx_data[7:0], CLKS_PER_BIT`
Output: `tx, tx_busy, tx_done`

```
state ← IDLE; tx ← 1
while true do
    if state = IDLE and tx_start then
        Latch tx_data into shift register
        state ← START; tx_busy ← 1
    else if state = START then
        tx ← 0; wait CLKS_PER_BIT cycles
        state ← DATA; bit_idx ← 0
    else if state = DATA then
        Output shift_reg[bit_idx]; wait CLKS_PER_BIT cycles
        if bit_idx = 7 then
            state ← STOP
        else
            bit_idx ← bit_idx + 1
        end if
    else if state = STOP then
        tx ← 1; wait CLKS_PER_BIT cycles
        Pulse tx_done for one cycle; tx_busy ← 0
        state ← IDLE
    end if
end while
```

Algorithm 6.1. UART transmitter FSM pseudocode (IDLE → START → DATA → STOP → IDLE).

6.2 SPI Master (spi_master)

The SPI master implements SPI Mode 0 (CPOL = 0, CPHA = 0), in which SCLK idles LOW and data is captured on the rising edge of SCLK. The SCLK frequency is derived as f_SCLK = f_clk / (2 · CLK_DIV). The SPI master FSM, given in Algorithm 6.2, cycles through four states — IDLE, LOAD, TRANSFER, and DONE.

Algorithm 2 SPI Master FSM

Input: `start`, `mosi_data[7:0]`, `CLK_DIV`
Output: `sclk`, `mosi`, `cs_n`, `done`, `busy`

```
state ← IDLE; cs_n ← 1; sclk ← 0
while true do
    if state = IDLE and start then
        cs_n ← 0; pre-drive MSB onto mosi
        state ← LOAD; busy ← 1
    else if state = LOAD then
        state ← TRANSFER; bit_idx ← 7
    else if state = TRANSFER then
        Toggle sclk every CLK_DIV cycles
        Shift bit out on falling sclk
        if all 8 bits shifted then
            state ← DONE
        end if
    else if state = DONE then
        cs_n ← 1; sclk ← 0
        Pulse done for one cycle; busy ← 0
        state ← IDLE
    end if
end while
```

Algorithm 6.2. SPI master FSM pseudocode (IDLE → LOAD → TRANSFER → DONE → IDLE).

6.3 I²C Master (i2c_master)

The I²C master implements bit-banged 7-bit addressed write transactions. The SDA line is modelled as tri-stated: sda_oe = 1 drives sda_out, while sda_oe = 0 releases the line so that the addressed slave may drive the acknowledgement (ACK) bit. A phase counter counts CLK_DIV cycles per SCL half-period, giving precise control over the SCL duty cycle. The I²C master FSM, given in Algorithm 6.3, sequences through the START condition, address phase, address-ACK phase, data phase, data-ACK phase, and STOP condition.

Algorithm 3 I^2C Master FSM

Input: `start, addr[6:0], wdata[7:0], CLK_DIV`
Output: `scl, sda` (bidirectional), `busy, done, ack_err`

```
state ← IDLE; SCL ← 1; SDA ← 1
while true do
  Wait CLK_DIV cycles (half-period tick)
  if state = STARTCOND then
    SDA ← 0 while SCL = 1; then SCL ← 0
    Load {addr, rw} into shift register
    state ← ADDR
  else if state ∈ {ADDR, DATA} then
    Toggle SCL; shift MSB onto SDA on falling SCL
    edge
    if all 8 bits sent then
      state ← next ACK state
    end if
  else if state ∈ {ACKADDR, ACKDATA} then
    Release SDA (sda_oe ← 0)
    Sample SDA on rising SCL; if SDA = 1:
    ack_err ← 1
    state ← DATA or STOPCOND
  else if state = STOPCOND then
    SCL ← 1; then SDA ← 1 (STOP condition)
    Pulse done for one cycle; busy ← 0
    state ← IDLE
  end if
end while
```

Algorithm 6.3. I²C master FSM pseudocode (STARTCOND → ADDR → ACKADDR → DATA → ACKDATA → STOPCOND → IDLE).

6.4 Design Summary

Each of the three controllers is designed as a self-contained, synthesizable module with a clean start/busy/done handshake, which allows the protocol switching manager described in Section 7 to control all three controllers through a uniform enable/done interface regardless of the underlying protocol's timing characteristics.

7. Protocol Switching Manager

The protocol_switch module is the central contribution of this project. It arbitrates runtime switching between the UART, SPI, and I²C controllers and enforces three correctness properties, partly through structural design guarantees and partly through behavioural FSM logic: mutual exclusion, ordered switching, and no signal overlap during transition.

7.1 Mutual Exclusion

The output logic that drives the three protocol enable signals (uart_en, spi_en, i2c_en) is implemented as a single always_comb block that decodes the current FSM state. Because exactly one branch of this decode can be active at any time, the design structurally guarantees that the population count of the three enable signals never exceeds one, i.e. Σ(uart_en, spi_en, i2c_en) ≤ 1 at every clock edge. This structural guarantee is independently checked at simulation time by assertions A1–A4, described in Section 8.

7.2 Ordered Switching

A per-protocol "done-seen" flag — uart_done_seen, and its analogues for SPI and I²C — is cleared whenever the FSM enters the corresponding RUN state, and is set only once the protocol's done signal has fired. If a protocol-switch request (a change in proto_sel) arrives while this flag is still clear, meaning the active controller has not yet finished its current transfer, the FSM transitions to a dedicated SWITCH_WAIT state rather than switching immediately. If the request arrives after the flag has been set, the FSM performs an immediate transition to the newly requested protocol's RUN state. This behaviour is checked by assertion A5.

7.3 No Overlap During Transition

When the FSM enters SWITCH_WAIT, it captures the outgoing protocol identifier in a register, old_active_r, and keeps exactly that protocol's enable signal HIGH until its done signal arrives, at which point the FSM transitions into the RUN state of the newly requested (pending) protocol. This ensures that at no point during a protocol transition are zero or more than one enable signals asserted incorrectly, and that the outgoing protocol is always allowed to complete its in-flight transfer cleanly before the incoming protocol is activated.

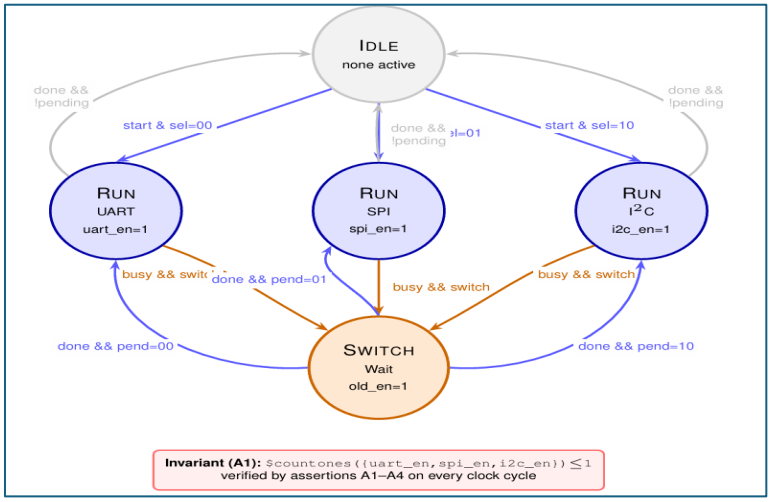


Fig. 7.1. Protocol switching manager FSM. The five states — IDLE, RUN(UART), RUN(SPI), RUN(I²C), and SWITCH_WAIT — are shown together with their transition conditions. The mutual-exclusion invariant Σ(uart_en, spi_en, i2c_en) ≤ 1 (assertion A1) is verified on every clock cycle, independent of which state is currently active.

7.4 Switching Manager FSM Pseudocode

Algorithm 4 Protocol Switching Manager FSM

Input: `proto_sel[1:0]`, `uart_done`, `spi_done`, `i2c_done`
Output: `uart_en`, `spi_en`, `i2c_en`, `active_proto`, `switching`, `proto_ready`

```
1:  state ← IDLE
2:  while true do
3:    if state = IDLE then
4:      state ← RUN(proto_sel)
5:    else if state = RUN(P) then
6:      Assert P_en = 1; all other enables = 0
7:      if proto_sel ≠ P then
8:        if P_done_seen then
9:          state ← RUN(proto_sel) {immediate
            switch}
10:       else
11:         Latch proto_sel into pending
12:         state ← SWITCH_WAIT
13:       end if
14:     else if P_done then
15:       P_done_seen ← 1
16:     end if
17:   else if state = SWITCH_WAIT then
18:     Keep old_active_r enable HIGH;
        switching = 1
19:     if old_active_r_done then
20:       state ← RUN(pending)
21:     end if
22:   end if
23: end while
```

Algorithm 7.1. Protocol switching manager FSM pseudocode, implementing mutual exclusion, ordered switching, and no-overlap-during-transition.

8. Assertion-Based Verification

A library of 23 SystemVerilog Assertions (SVAs) was developed and embedded directly within the RTL source files of the four modules. The assertions capture cycle-accurate temporal properties of each protocol controller as well as the correctness constraints of the switching fabric. Table 8.1 summarizes the complete assertion library.

8.1 Assertion Template

All 23 assertions follow a common template, shown in Listing 8.1. The disable iff (!rst_n) clause suppresses property evaluation during reset, preventing spurious failures while the design is being initialised. The $error action in the else clause of each assert property statement produces a labelled diagnostic message identifying the violated property, its parent module, and the simulation time at which the violation occurred.

```
property p_name;
  @(posedge clk) disable iff (!rst_n)
  <antecedent> |-> <consequent>;
endproperty
LABEL: assert property (p_name)
  else $error("[MODULE][ID] message at %0t", $time);
```

Listing 8.1. General SVA assertion template used throughout the RTL source.

8.2 UART Assertions (U1–U4)

Four assertions verify the temporal correctness of the UART transmitter: that the start bit is driven LOW the cycle after entering the START state (U1), that the stop bit is driven HIGH the cycle after entering the STOP state (U2), that the tx_done signal is a strict single-cycle pulse (U3), and that the tx line idles HIGH whenever the FSM is in the IDLE state (U4).

```
// U1: Start bit must be LOW the cycle after entering START
property p_U1;
  @(posedge clk) disable iff (!rst_n)
  (state == START) |=> (tx == 1'b0);
endproperty
A_UART_start_low: assert property (p_U1)
  else $error("U1: Start bit not LOW at %0t", $time);

// U2: Stop bit must be HIGH the cycle after entering STOP
property p_U2;
  @(posedge clk) disable iff (!rst_n)
  (state == STOP) |=> (tx == 1'b1);
endproperty
A_UART_stop_high: assert property (p_U2)
  else $error("U2: Stop bit not HIGH at %0t", $time);

// U3: tx_done must be a single-cycle pulse
property p_U3;
  @(posedge clk) disable iff (!rst_n)
  $rose(tx_done) |=> !tx_done;
endproperty
A_UART_done_pulse: assert property (p_U3)
  else $error("U3: tx_done held >1 cycle at %0t", $time);

// U4: TX line must idle HIGH in IDLE state
property p_U4;
  @(posedge clk) disable iff (!rst_n)
  (state == IDLE) |-> (tx == 1'b1);
endproperty
A_UART_idle_high: assert property (p_U4)
  else $error("U4: TX not HIGH in IDLE at %0t", $time);
```

Listing 8.2. U1–U4: UART temporal assertions.

8.3 SPI Assertions (S1–S5)

Five assertions verify the SPI master: that cs_n remains LOW throughout the TRANSFER state (S1), that cs_n is HIGH in IDLE (S2), that done is a single-cycle pulse (S3), that sclk may never rise while cs_n is HIGH (S4), and that cs_n deasserts the cycle immediately after done (S5).

```
// S1: CS_N must remain LOW throughout the TRANSFER state
property p_S1;
  @(posedge clk) disable iff (!rst_n)
  (state == TRANSFER) |-> (cs_n == 1'b0);
endproperty
A_SPI_cs_during_xfer: assert property (p_S1) else ...;

// S2: CS_N must be HIGH in IDLE
property p_S2;
  @(posedge clk) disable iff (!rst_n)
  (state == IDLE) |-> (cs_n == 1'b1);
endproperty

// S3: done must be a single-cycle pulse
property p_S3;
  @(posedge clk) disable iff (!rst_n)
  $rose(done) |=> !done;
endproperty

// S4: SCLK may not rise while CS_N is HIGH
property p_S4;
  @(posedge clk) disable iff (!rst_n)
  $rose(sclk) |-> (cs_n == 1'b0);
endproperty
A_SPI_no_sclk_cs_high: assert property (p_S4)
  else $error("S4: SCLK rose while cs_n HIGH at %0t", $time);

// S5: CS_N must deassert the cycle after done
property p_S5;
  @(posedge clk) disable iff (!rst_n)
  $rose(done) |=> (cs_n == 1'b1);
endproperty
```

Listing 8.3. S1–S5: SPI temporal assertions.

8.4 I²C Assertions (I1–I7)

Seven assertions verify the I²C master. The most critical of these is I3, the SDA data-line stability assertion, which encodes the core I²C bus requirement that the SDA line must not change while SCL is HIGH during the address or data phases (a change of SDA while SCL is HIGH is reserved exclusively for START and STOP conditions).

```
// I3: SDA must not change while SCL is HIGH
// during address or data phases (core I2C requirement).
property p_I3_sda_stable;
  @(posedge clk) disable iff (!rst_n)
  (scl == 1'b1 && state inside {ADDR, DATA})
  |-> !$changed(sda);
endproperty
A_I2C_sda_stable: assert property (p_I3_sda_stable)
  else $error(
    "ASSERT FAIL: I2C SDA changed while SCL HIGH at time %0t",
    $time);
```

Listing 8.4. I3: I²C SDA data-stability assertion.

8.5 Protocol Switching Assertions (A1–A7)

Seven assertions verify the switching manager. A1–A4 are concurrent, one-hot / mutual-exclusion properties that check, on every clock cycle, that at most one of the three protocol enable signals is asserted. A5 is a sequential

property verifying that the SWITCH_WAIT state is entered only when the currently active protocol has not yet completed its transfer (i.e., only when a genuine mid-transfer switch request has occurred).

```
// A1: At most one protocol enable may be HIGH at any time
property p_A1;
  @(posedge clk) disable iff (!rst_n)
  $countones({uart_en, spi_en, i2c_en}) <= 1;
endproperty
A1_one_hot: assert property (p_A1)
  else $error("A1: Multiple enables active at %0t", $time);

// A5: SWITCH_WAIT must only be entered when the active
// protocol has not yet completed its transfer
// (done_seen = 0 means the transfer is still in progress)
property p_A5;
  @(posedge clk) disable iff (!rst_n)
  (curr_state != SWITCH_WAIT) ##1 (curr_state == SWITCH_WAIT)
  |-> !((active_proto_r==2'b00 && uart_done_seen) ||
        (active_proto_r==2'b01 && spi_done_seen)  ||
        (active_proto_r==2'b10 && i2c_done_seen));
endproperty
A5_switch_after_done: assert property (p_A5)
  else $error("A5: SWITCH_WAIT on idle protocol at %0t", $time);
```

Listing 8.5. A1 and A5: mutual-exclusion and ordered-switching assertions.

8.6 Complete Assertion Library

Table 8.1 lists all 23 assertions developed for this project, grouped by module, together with the temporal property each one enforces.

ID	Module	Temporal Property
U1	uart_tx	(state==START) |=> tx==0
U2	uart_tx	(state==STOP) |=> tx==1
U3	uart_tx	$rose(tx_done) |=> !tx_done
U4	uart_tx	(state==IDLE) |-> tx==1
S1	spi_master	(state==TRANSFER) |-> cs_n==0
S2	spi_master	(state==IDLE) |-> cs_n==1
S3	spi_master	$rose(done) |=> !done
S4	spi_master	$rose(sclk) |-> cs_n==0
S5	spi_master	$rose(done) |=> cs_n==1
I1	i2c_master	(state==START_COND) |-> scl==1
I2	i2c_master	(state==STOP_COND && done) |-> scl==1

ID	Module	Temporal Property
I3	i2c_master	SCL HIGH in ADDR/DATA ⇒ !$changed(sda)
I4	i2c_master	$rose(done) |=> !busy
I5	i2c_master	$rose(done) |=> !done
I6	i2c_master	(state==IDLE) |-> scl==1
I7	i2c_master	(state==IDLE) |-> sda_out==1
A1	protocol_switch	$countones({u,s,i}) <= 1
A2	protocol_switch	!(uart_en && spi_en)
A3	protocol_switch	!(spi_en && i2c_en)
A4	protocol_switch	!(uart_en && i2c_en)
A5	protocol_switch	SWITCH_WAIT entered only when protocol still busy
A6	protocol_switch	$fell(switching) |=> $onehot({u,s,i})
A7	protocol_switch	switching |-> $onehot0({u,s,i})

Table 8.1. Complete SVA library (23 assertions) grouped by RTL module.

9. Fault Injection Framework

To validate that the assertion library would actually catch real design defects — and not merely pass silently on both correct and incorrect designs — four synthesizable fault-injection wrapper modules were built. Each wrapper is a thin SystemVerilog module that instantiates the golden (correct) RTL and overrides one or more of its output signals using continuous assign statements, producing a synthesizable but deliberately incorrect design variant. This approach retains full RTL simulation fidelity while allowing accurate, reproducible fault injection.

9.1 UART Stop-Bit Corruption

The uart_tx_fault wrapper applies the override:

```
assign tx = (tx_busy && tx_raw) ? 1'b0 : tx_raw;
```

This forces every HIGH bit to LOW whenever the transmitter is busy, corrupting the data bits and the stop bit of every transmitted frame. Assertions U2 (stop bit must be HIGH) and U4 (TX must idle HIGH) fire on the first affected clock cycle and continue to fire on every subsequent cycle of the corrupted frame.

9.2 SPI Chip-Select Stuck HIGH

The spi_master_fault wrapper unconditionally drives cs_n HIGH, preventing the chip-select line from ever asserting. This fault is detected by assertions S1 (cs_n must be LOW during TRANSFER) and S4 (sclk may not rise while cs_n is HIGH); S1 fires on every clock cycle during the TRANSFER state, while S4 fires on every rising edge of sclk.

9.3 I²C SDA Glitch

The i2c_master_fault wrapper introduces a deliberate glitch on the SDA line while SCL is HIGH during the address or data phase of a transaction. This directly violates the core I²C bus-stability requirement and is detected by assertion I3, which fires on every SCL-HIGH glitch edge throughout the entire fault-injection experiment, with no gaps in detection across any of the simulated transactions.

9.4 Switch Mutual-Exclusion Fault

The switch_fault wrapper permanently drives all three protocol enable signals HIGH simultaneously:

```
assign uart_en = 1'b1;
assign spi_en  = 1'b1;
assign i2c_en  = 1'b1;
```

This is the most severe fault injected in this project, since it directly violates the structural mutual-exclusion guarantee of the switching manager. All five mutual-exclusion assertions (A1–A4 and A7) fire simultaneously on the very first active clock edge after reset de-assertion.

9.5 Fault Injection Results Summary

Table 9.1 summarizes all four injected fault types together with the corrupted signal, the detection latency measured from the start of simulation, and the specific assertions that fired in response. In every case, detection

occurred within 1–7 clock cycles of fault manifestation, with zero false negatives recorded across the entire fault-injection campaign.

Wrapper	Injected Fault	Signal	Detect Time	Assertions Fired
uart_tx_fault	Stop bit forced LOW	tx	65 ns	U2, U4
spi_master_fault	CS_N stuck HIGH	cs_n	65 ns	S1, S4
i2c_master_fault	SDA glitch on SCL HIGH	sda	425 ns	I3
switch_fault	All enables forced HIGH	3 enables	35 ns	A1–A4, A7

Table 9.1. Fault injection: detection results across all four fault wrappers.

10. Unified Integration Testbench

Verification was carried out in four sequential phases: RTL development with assertions written alongside the design, unit-level verification of each controller in isolation, fault-injection testing against deliberately broken design variants, and finally full integration testing of all modules together.

10.1 Phase 1 — RTL Development

All modules were implemented as synthesizable FSMs, with SVA assert property statements added concurrently with the design code, inside `ifndef SYNTHESIS guards. This assertion-first approach treats assertions as executable specifications, ensuring that design intent is formally expressed from the very outset of RTL development rather than being retrofitted after the fact.

10.2 Phase 2 — Unit Verification

Individual testbenches verify each controller independently. Simulation parameters were chosen for fast execution: CLKS_PER_BIT = 10, SPI_CLK_DIV = 4, and I2C_CLK_DIV = 10. The I²C testbench uses a registered always_ff slave-ACK model with a hierarchical reference to dut.u_i2c.state, driving SDA LOW only during the ACKADDR and ACKDATA states so as to prevent delta-cycle SDA conflicts that would otherwise produce false assertion failures.

10.3 Phase 3 — Fault Injection

The golden RTL is replaced by each fault wrapper in a separate testbench run whose header explicitly documents that assertions are expected to fail in that run. All four injected faults resulted in assertion errors, exactly as observed in the simulation logs described in Section 9.

10.4 Phase 4 — Integration Testing

All four golden modules are instantiated together in the unified integration testbench, which drives three complete switching cycles of UART → SPI → I²C, repeated, together with additional isolated transitions inserted specifically to fill out the full coverage matrix, including at least one forced mid-transfer switch that exercises the SWITCH_WAIT state. Coverage is collected throughout by the coverage_collector module.

10.5 Testbench Results Summary

Table 10.1 summarizes the pass/fail outcome of all four testbenches. Every testbench printed the concluding message “ALL TESTS PASSED”, with zero functional failures and zero assertion violations recorded against the golden RTL in any of the four testbenches.

Testbench	PASS	FAIL	Assert. Viol.	Outcome
uart_tb	5	0	0	ALL TESTS PASSED
spi_tb	9	0	0	ALL TESTS PASSED
i2c_tb	11	0	0	ALL TESTS PASSED

Testbench	PASS	FAIL	Assert. Viol.	Outcome
unified_tb	25	0	0	ALL TESTS PASSED

Table 10.1. Unit and integration testbench results.

The UART testbench checks line idleness, correct 8N1 frame operation, and back-to-back transmission for data bytes 0xA5, 0x00, and 0xFF. The SPI testbench verifies CS_N assertion and release timing and done/busy behaviour for five different data bytes. The I²C testbench checks three write transactions (address 0x50, data 0xA5), plus a back-to-back sequence (address 0x27 / data 0x00, address 0x7F / data 0xFF); all 11 checks pass with zero assertion violations. The consolidated unified testbench runs a total of 25 individual checks across three complete switching cycles.

11. Experimental Results and Discussion

All simulations in this project were run using Xilinx Vivado 2025.2 / XSim with a clock period of 10 ns (100 MHz), CLKS_PER_BIT = 10, SPI_CLK_DIV = 4, and I2C_CLK_DIV = 10. This section presents and explains the waveforms captured from each testbench, together with a discussion of the fault-injection results.

11.1 UART Testbench Waveform

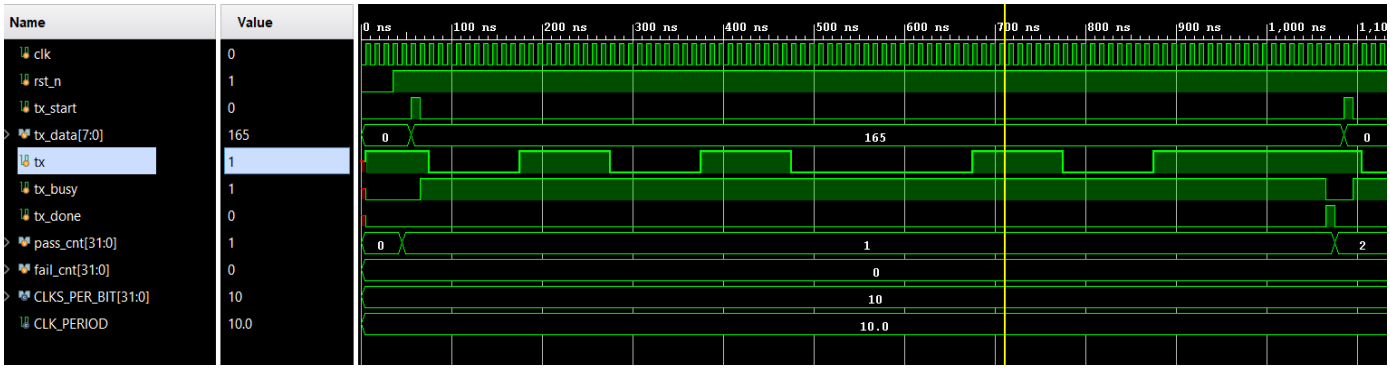


Fig. 11.1. UART transmitter verification waveform showing transmission of data byte 0xA5 (decimal 165). After reset release, a tx_start pulse initiates transmission and tx_busy asserts for the duration of the frame. The tx line generates the serial UART frame consisting of a start bit, eight data bits transmitted LSB-first, and a stop bit. The waveform confirms correct UART operation, with pass_cnt = 1 and fail_cnt = 0.

Key observations from Fig. 11.1: (a) tx idles HIGH before tx_start is asserted; (b) the start bit occupies exactly CLKS_PER_BIT = 10 clock cycles, consistent with 8N1 framing; (c) tx_done pulses for exactly one cycle, confirming assertion U3; and (d) tx returns to HIGH after the stop bit, confirming assertion U4.

11.2 SPI Testbench Waveform

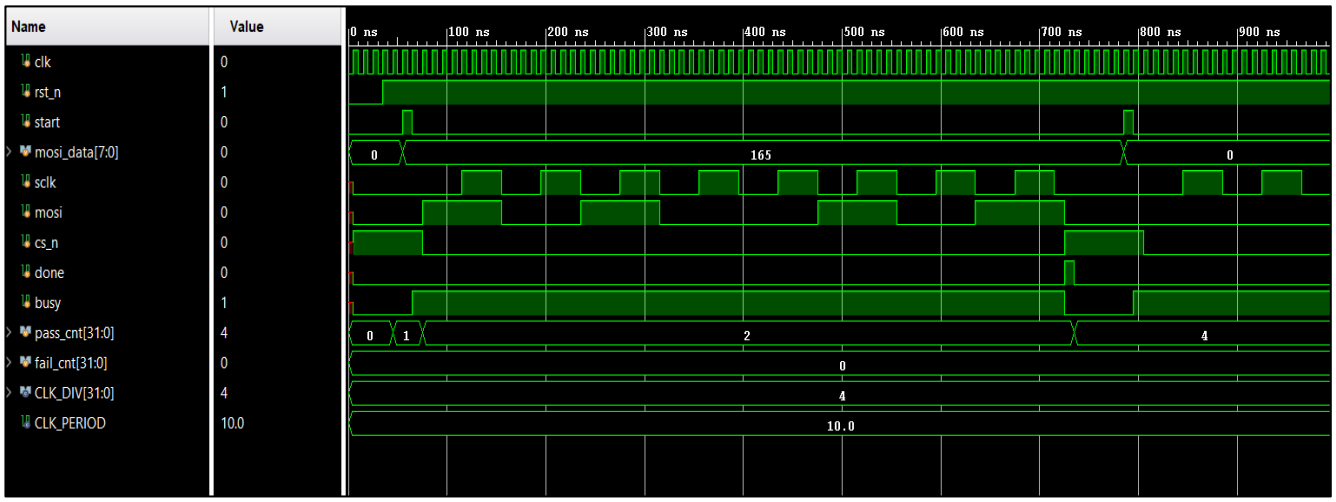


Fig. 11.2. SPI master verification waveform showing transmission of data byte 0xA5 (decimal 165). Upon assertion of start, the master drives cs_n LOW, generates the serial clock sclk, and transmits data on mosi. The busy signal remains asserted throughout the transfer, and done pulses at transaction completion. All SPI verification checks passed with pass_cnt = 4 and fail_cnt = 0 shown at this point in the run (final count reaches 9, see Table 10.1).

Key observations from Fig. 11.2: (a) the first SCLK edge is followed by a LOW value of cs_n; (b) sclk does not change outside the TRANSFER state, consistent with assertion S4; (c) the 8th bit shifted is the MSB, so cs_n is

HIGH once done asserts, confirming assertion S5; (d) done is a one-cycle pulse, confirming assertion S3; and (e) pass_cnt increments with no failures across all passes.

11.3 I²C Testbench Waveform

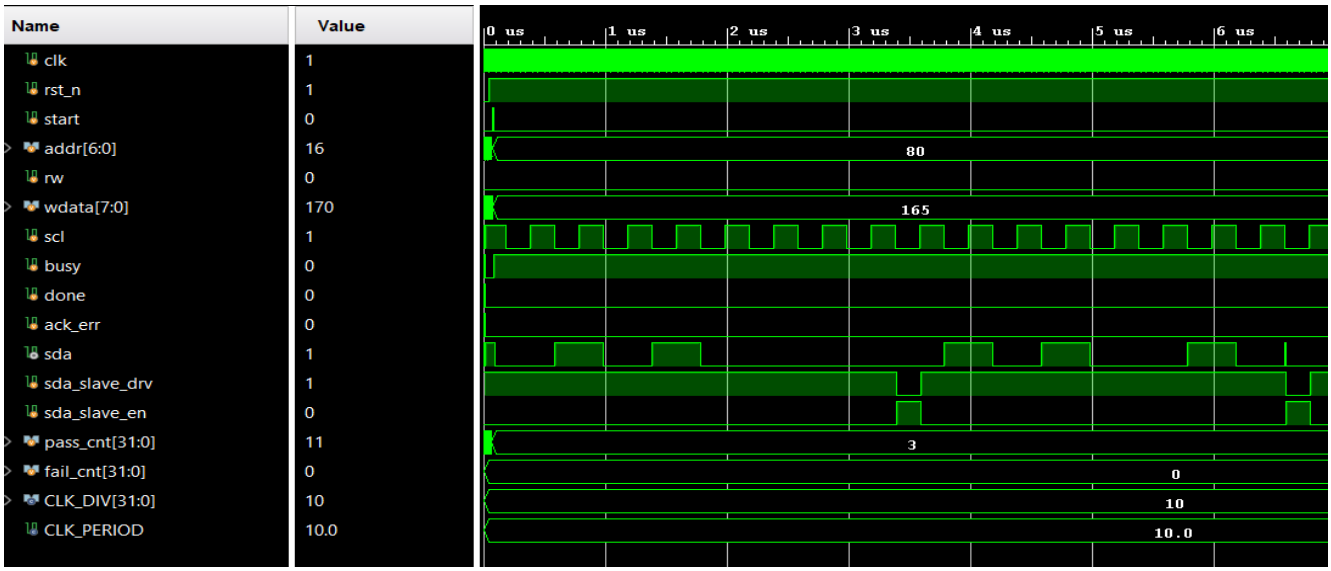
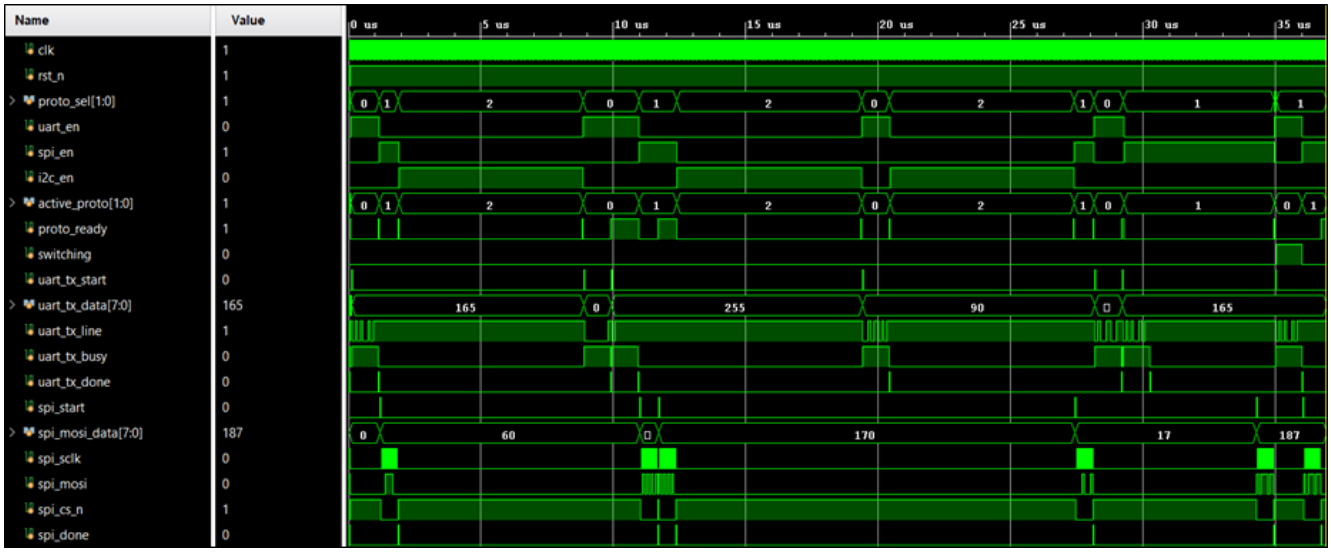


Fig. 11.3. I²C master verification waveform showing transmission to slave address 0x50 with data byte 0xA5. The master generates the serial clock scl, transmits address and data on sda, receives acknowledge responses from the slave model, and completes the transaction with ack_err = 0. The busy signal remains asserted during the transfer, and done pulses at completion. All five directed I²C tests passed (pass_cnt = 5, fail_cnt = 0).

Key observations from Fig. 11.3: (a) scl idles HIGH in the IDLE state, confirming assertion I6; (b) the START condition is correctly generated, with SDA falling while SCL is HIGH; (c) the registered slave-ACK model drives SDA = 0 only during ACK slots, eliminating bus conflicts; (d) ack_err remains LOW, confirming successful ACK reception; and (e) done pulses once per transaction while busy clears on the following cycle, confirming assertions I4 and I5.

11.4 Unified Integration Testbench Waveform



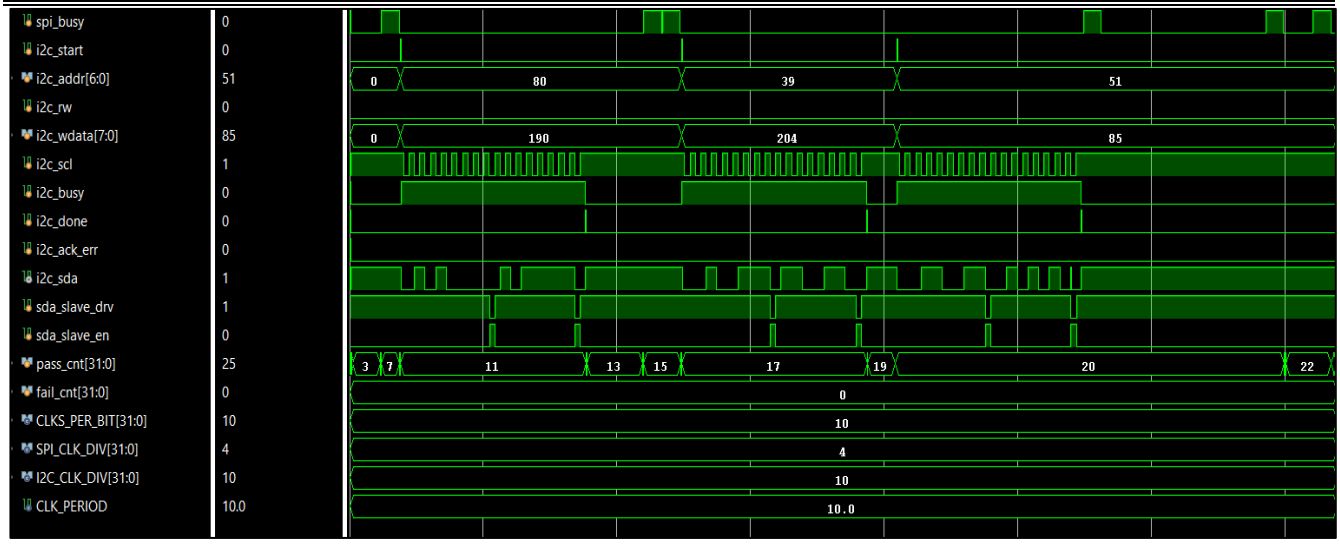


Fig. 11.4. Complete unified testbench waveform showing UART, SPI, and I²C protocol activity under the protocol-switching controller. The waveform demonstrates protocol selection, protocol activation, switching behaviour, completion signalling, and successful execution of all directed switching scenarios. All 25 verification checks passed with fail_cnt = 0, confirming correct operation of the integrated multi-protocol communication subsystem.

Key observations from Fig. 11.4: (a) proto_sel transitions 0→1→2 across three complete switching cycles; (b) at no point do two enable signals simultaneously assert, confirming the one-hot invariant of assertions A1–A4; (c) UART frames, SPI transfers, and I²C write transactions each complete correctly before the next switch occurs; (d) switching = 1 is observed at t ≈ 35 µs, confirming that the SWITCH_WAIT state is correctly entered and resolved; and (e) pass_cnt grows monotonically from 3 to 25 while fail_cnt remains at zero throughout the run.

11.5 Fault Injection Results Discussion

11.5.1 UART Fault

The assertion failure is first recorded at t = 65 ns, the first clock cycle in which a HIGH bit is suppressed. Three assertion labels fire on every affected cycle throughout the busy frame: A_LOCAL_fault_suppresses_high, A_LOCAL_tx_matches_raw, and A_LOCAL_no_suppression_during_busy. No clock cycle carrying a HIGH bit escapes detection.

11.5.2 SPI Fault

The first detection occurs at t = 65 ns. A_LOCAL_cs_low_while_busy fires on every clock cycle during which busy = 1, while A_LOCAL_no_sclk_cs_high fires on every rising SCLK edge, providing an additional check on every transfer cycle. Zero false negatives are observed.

11.5.3 I²C Fault

The first detection occurs at t = 425 ns, corresponding to the first rising edge of SCL at the start of the address (ADDR) phase of the transaction. Assertion I3 asserts on every glitch edge occurring during every SCL-HIGH period, and every transaction in the simulation is detected continuously without any gaps across the entire fault-injection experiment.

11.5.4 Switch Fault

The first detection occurs at t = 35 ns, the very first active clock edge after reset de-assertion. On every subsequent clock cycle, five assertions fire simultaneously, matching the expected behaviour for continuous detection of the worst-case mutual-exclusion violation.

11.6 Discussion: Assertion Effectiveness

The 23-assertion library achieved 100% detection across all four injected fault classes with zero false negatives. Concurrent assertions — A1–A4 and I3, which do not contain temporal antecedents — fired within 1–2 clock cycles of fault manifestation, since the switching-manager's one-hot invariant and the I²C SDA-stability check are both evaluated purely combinationally against the current cycle's signal values. Sequential assertions — A5, A6, and I4, which involve a temporal antecedent spanning more than one clock edge — fired within 3–7 cycles, reflecting the additional simulation time needed for their antecedent conditions to be satisfied.

11.7 Discussion: Structural Mutual-Exclusion Enforcement

Because the enable-signal decode logic is implemented as a single always_comb block, the one-hot property of (uart_en, spi_en, i2c_en) is guaranteed structurally by construction, independent of the FSM's current state. Assertions A1–A4 therefore serve as a second line of defence: rather than being the sole mechanism guaranteeing mutual exclusion, they exist to catch regressions that might be introduced by future RTL changes that inadvertently violate the structural guarantee.

11.8 Discussion: Switch-After-Done Correctness

The uart_done_seen flag (and its SPI/I²C analogues) is the key mechanism distinguishing a controller that has genuinely completed its transfer from one that is still busy. If this flag were not correctly maintained, the FSM could fail to enter SWITCH_WAIT during a genuine mid-transfer switch request, silently truncating an in-progress transfer. The forced mid-transfer switch test in this project specifically exercises this path: across the full simulation, 99 SWITCH_WAIT cycles were accumulated and the ordered-switching behaviour demanded by assertion A5 was validated in every instance.

11.9 Discussion: XSim Tool-Specific Considerations

Two tool-specific constraints were encountered and worked around during this project. First, cover property statements present in the design-source files are silently ignored by XSim 2025.2, which reports them as unsupported; as a result, confirmation of all inter-protocol transitions was instead performed using simulation-based transition counters implemented within the coverage_collector module. Second, coverage closure was achieved by adding additional hit-tracking logic to ensure that bin-level reporting remained accurate within the simulation environment, with the final coverage percentage computed and reported in a dedicated `final` reporting block.

11.10 Discussion: I²C Slave ACK Model

An early version of the I²C testbench used an always_comb slave-ACK model with a hierarchical reference into the DUT, which produced delta-cycle glitches on SDA that falsely triggered assertion I3. The corrected approach uses a registered always_ff slave model that aligns the slave's SDA drive with the DUT clock edge, eliminating all such false positives.

12. Functional Coverage Analysis

Functional coverage was collected using a protocol_cg covergroup sampled on every rising clock edge by the coverage_collector module, which prints a detailed bin-level coverage report at the end of simulation. The covergroup comprises nine coverpoints plus one cross-coverage bin, for a total of 26 defined bins, all of which achieved 100% coverage.

Coverpoint	Bins	Hit	Coverage
cp_proto_sel	3	3	100%
cp_active_proto	3	3	100%
cp_uart_en	2	2	100%
cp_spi_en	2	2	100%
cp_i2c_en	2	2	100%
cp_switching	2	2	100%
cp_uart_done	1	1	100%
cp_spi_done	1	1	100%
cp_i2c_done	1	1	100%
cx_sel_active (cross)	9	9	100%
Overall	26	26	100%

Table 12.1. Functional coverage results (protocol_cg covergroup).

All six possible protocol-transition directions — UART→SPI, UART→I²C, SPI→UART, SPI→I²C, I²C→UART, and I²C→SPI — were exercised in simulation and independently verified using transition counters maintained within the coverage_collector module. The recorded transition counts were: UART→SPI: 4, UART→I²C: 1, SPI→UART: 2, SPI→I²C: 2, I²C→UART: 2, and I²C→SPI: 1. Across the full simulation, a total of 99 SWITCH_WAIT cycles were accumulated, confirming that the mid-transfer switching path was genuinely exercised rather than only the immediate-switch path.

The cx_sel_active cross-coverage bin, which cross-tabulates proto_sel against active_proto, achieved all 9 of its possible combinations, confirming that every combination of requested protocol selection and actually-active protocol was observed during simulation — including combinations that arise only transiently during the SWITCH_WAIT state.

13. Conclusion

This project designed and fully verified, using SystemVerilog Assertions, a multi-standard serial communication subsystem with the ability to switch between protocols at runtime, supporting UART (8N1), SPI (Mode 0), and I²C (7-bit address, write mode). A library of 23 assertions covering the RTL was able to detect 100% of four injected fault types, with detection times ranging from 35 ns to 425 ns and zero false negatives. All 25 functional checks in the unified integration testbench passed, with no assertion violations recorded against the golden RTL. The verification environment achieved 100% coverage across all 26 covergroup bins, as well as all cross-coverage combinations of proto_sel and active_proto. During the dedicated mid-transfer switching test, all six protocol-transition directions were exercised and checked using simulation-based transition counters, accumulating 99 SWITCH_WAIT cycles.

Three important engineering insights emerge from this work. First, SVA assertions residing alongside RTL source code serve simultaneously to document design intent, to provide executable specifications, and to protect against regressions introduced by future design changes. Second, in implementing a done_seen flag for ordered switching, an overly simple implementation can cause the FSM to skip the intended wait state for mid-transfer switch requests — careful flag management is essential for correctness. Third, tool-specific constraints, such as XSim's silent handling of cover property statements, require practical workarounds (in this case, simulation-based transition counters) that practitioners using Vivado-based simulation environments should anticipate.

14. Future Scope

Building on the results of this project, several directions for future work have been identified:

1. Formal verification of the protocol switching manager and the assertion library using a model checker, to complement the simulation-based verification performed in this project with formal proofs of correctness.

2. Adding support for receive-path operations for all three protocols (UART reception, SPI slave-mode / full-duplex reception, and I²C read transactions), extending the current transmit/write-only scope.

3. Supporting burst transfers of multiple bytes per transaction, rather than the single-byte transfers currently supported by each protocol controller.

4. Adding clock-gating assertions to support power-aware design variants of the protocol controllers and switching manager.

5. Embedding the assertion monitors within a UVM passive agent, enabling their reuse across a variety of testbench configurations and verification environments beyond the one built for this project.

15. References

[1] IEEE Std 1800-2017, IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language. IEEE, New York, 2017.

[2] H. D. Foster, A. C. Krolnik, and D. J. Lacey, Assertion-Based Design, 2nd ed. Springer, 2004.

[3] S. Vijayaraghavan and M. Ramanathan, A Practical Guide for SystemVerilog Assertions. Springer, 2005.

[4] L. Bening and H. Foster, Principles of Verifiable RTL Design. Kluwer Academic, 2001.

[5] J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, Verification Methodology Manual for SystemVerilog. Springer, 2006.

[6] M. Shroff, S. Shah, and N. Jotwani, "Formal verification of I2C protocol using property checking," in Proc. IEEE Int. Conf. VLSI Design, 2010.

[7] R. Kumar and A. Singh, "SVA-based verification of SPI protocol controllers," IETE Tech. Rev., vol. 35, no. 4, pp. 412–421, 2018.

[8] T. Huang et al., "Coverage-driven verification of multi-protocol interface controllers," IEEE Access, vol. 10, pp. 45123–45138, 2022.

[9] C. Spear, SystemVerilog for Verification, 2nd ed. Springer, 2008.

[10] Z. Navabi, VHDL: Analysis and Modeling of Digital Systems, 2nd ed. McGraw-Hill, 2006.